From Basic CUDA to Kimi Kernels

Deep Dive into Kimi K3 — Chapter 32

Yin Yang

Foundation Books Project

The one idea of this chapter

A kernel is correct only when four maps agree:
who computes a value, where it lives, when it is ready, and when it may be reused.

The one idea of this chapter

A kernel is correct only when four maps agree:
who computes a value, where it lives, when it is ready, and when it may be reused.

Goal: follow a cooperative warp reduction and a 2×2 tile from logical coordinates to addresses, compiled instructions, CuTE partitions, and a double-buffered pipeline — with exact oracles and declared timing boundaries.

Roadmap

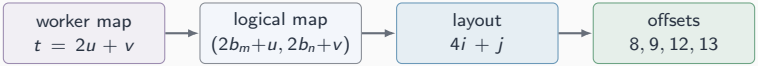
Workers, tiles, and layouts

From source to instructions and partitions

Readiness, reuse, and validation

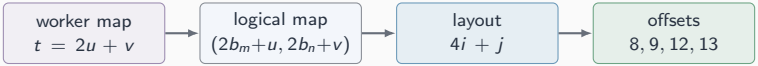
Workers, tiles, and layouts

Cooperation needs participants and agreed maps



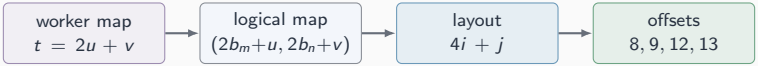
- One warp, one output: 32 private sums and five XOR exchanges give 8256; a data predicate is not a participant mask.

Cooperation needs participants and agreed maps



- One warp, one output: 32 private sums and five XOR exchanges give 8256; a data predicate is not a participant mask.
- Two slices fill one accumulator: $D[2{:}4, 0{:}2] = \left[\begin{smallmatrix} 50 & 60 \\ 12 & 16 \end{smallmatrix}\right]$; 16 staged operand elements versus 32 independent ones (logical counts).

Cooperation needs participants and agreed maps



- One warp, one output: 32 private sums and five XOR exchanges give 8256; a data predicate is not a participant mask.
- Two slices fill one accumulator: $D[2{:}4, 0{:}2] = \left[\begin{smallmatrix} 50 & 60 \\ 12 & 16 \end{smallmatrix}\right]$; 16 staged operand elements versus 32 independent ones (logical counts).
- An XOR swizzle spreads a bank-0 column over 32 banks, if producer and consumer share the map. [DERIVED: constructed fixtures]

From source to instructions and partitions

Legal instructions, composed views, separate partitions

- CUDA C++ $\rightarrow$ PTX $\rightarrow$ target device code; CUTLASS and CuTE construct the source. Legal instructions can still compute 53 instead of 50.

Legal instructions, composed views, separate partitions

- CUDA C++ $\rightarrow$ PTX $\rightarrow$ target device code; CUTLASS and CuTE construct the source. Legal instructions can still compute 53 instead of 50.
- Matrix instructions prescribe fragments and participants; a backend name is not an execution record.

Legal instructions, composed views, separate partitions

- CUDA C++ $\rightarrow$ PTX $\rightarrow$ target device code; CUTLASS and CuTE construct the source. Legal instructions can still compute 53 instead of 50.
- Matrix instructions prescribe fragments and participants; a backend name is not an execution record.
- CuTE: a layout is a coordinate function, a view is not a copy, and `local_partition` chooses who moves each value; worker 2 copies offset 12 to 2. [OPEN: companion CUDA programs not yet compiled or run]

Readiness, reuse, and validation

Ready is not reusable; correct is not fast

- Two slots, $L = 2$, $C = 3$: S0 is readable at 2 but reusable only at 5; three slices finish at 11 versus 15 serially. [DERIVED: schedule, not latency]

Ready is not reusable; correct is not fast

- Two slots, $L = 2$, $C = 3$: S0 is readable at 2 but reusable only at 5; three slices finish at 11 versus 15 serially. [DERIVED: schedule, not latency]
- A slot needs a generation; TMA readiness and WGMMA completion are separate waits, and overlap spends storage.

Ready is not reusable; correct is not fast

- Two slots, $L = 2$, $C = 3$: S0 is readable at 2 but reusable only at 5; three slices finish at 11 versus 15 serially. [DERIVED: schedule, not latency]
- A slot needs a generation; TMA readiness and WGMMA completion are separate waits, and overlap spends storage.
- Validate values *and* places; time the declared path: $5 + 8 = 13$, not 8. [DERIVED: constructed timing example]

Ready is not reusable; correct is not fast

- Two slots, $L = 2$, $C = 3$: S0 is readable at 2 but reusable only at 5; three slices finish at 11 versus 15 serially. [DERIVED: schedule, not latency]
- A slot needs a generation; TMA readiness and WGMMA completion are separate waits, and overlap spends storage.
- Validate values *and* places; time the declared path: $5 + 8 = 13$, not 8. [DERIVED: constructed timing example]

These maps recur in Chapters 34–37. Next: Chapter 33's LLM Parallelism Foundations, from one GPU to a distributed model.